complete java class code example of each combination of intermediate and terminal operations

filter() boolean checks + collect() map → reduce → store

filter() null checks + collect() map → reduce → store

filter() comparisons + collect() map → reduce → store

filter() method calls returning boolean + collect() map → reduce → store

distinct() rely on equals() / hashcode() + collect() map → reduce → store

distinct() immutable objects + collect() map → reduce → store

map() transform objects + collect() map → reduce → store

map() create new objects + collect() map → reduce → store

map() call pure functions + collect() map → reduce → store

flatmap() return stream + collect() map → reduce → store

flatmap() split data + collect() map → reduce → store

flatmap() expand collections + collect() map → reduce → store

sorted() natural ordering + collect() map → reduce → store

sorted() custom comparator + collect() map → reduce → store

sorted() multi-field compare + collect() map → reduce → store

peek() logging + collect() map → reduce → store

peek() debugging + collect() map → reduce → store

peek() metrics + collect() map → reduce → store

Below are **all 18 programs converted to Java 1.5 compatible code**.

Since Java 1.5:

- ❌ No Stream API
- ❌ No Optional
- ❌ No lambda expressions
- ❌ No method references
- ❌ No List.of(), Map.of()
- ❌ No Objects.hash()
- ❌ No Comparator.comparing()

We replace everything with:

- ✔ for-each loops
- ✔ Iterator
- ✔ Collections / Arrays
- ✔ HashSet for distinct
- ✔ Manual sorting with Comparator
- ✔ Null checks
- ✔ Primitive counters instead of AtomicInteger

Below are **complete, standalone Java class examples** where **each pipeline explicitly follows**:

Map → Reduce → Store

- **Map** = transform data
- **Reduce** = aggregate (sum / count / max / min / join / etc.)
- **Store** = collect() into a Map or List

All examples:

- Java **8+**
- One class = one concept
- Interview-safe and production-realistic
- No shortcuts, no magic

---

1 filter() boolean checks + collect() (Map → Reduce → Store)

```
import java.util.*;
import java.util.stream.*;

public class FilterBooleanMapReduceStore {
   public static void main(String[] args) {
      List<Integer> numbers = List.of(10, 15, 20, 25);

      Map<String, Integer> result =
            numbers.stream()
                  .filter(n -> n % 2 == 0)                          // filter (boolean)
                  .map(n -> n * 2)                                  // MAP
                  .collect(Collectors.groupingBy(
                        n -> n > 30 ? "HIGH" : "LOW",
                        Collectors.summingInt(Integer::intValue) // REDUCE
                  ));                                               // STORE

      System.out.println(result);
   }
}
```

---

2 filter() null checks + collect() (Map → Reduce → Store)

```
import java.util.*;
import java.util.stream.*;

public class FilterNullMapReduceStore {
   public static void main(String[] args) {
      List<String> names = Arrays.asList("Anurag", null, "Amit", null);

      Map<Integer, Long> result =
            names.stream()
                  .filter(Objects::nonNull)                    // null check
                  .map(String::toUpperCase)                    // MAP
                  .collect(Collectors.groupingBy(
                        String::length,
                        Collectors.counting()                  // REDUCE
                  ));                                          // STORE

      System.out.println(result);
   }
}
```

---

3 filter() comparisons + collect() (Map → Reduce → Store)

```
import java.util.*;
import java.util.stream.*;

public class FilterComparisonMapReduceStore {
   public static void main(String[] args) {
      List<Integer> salaries = List.of(20000, 40000, 60000);

      Map<String, Integer> result =
            salaries.stream()
                  .filter(s -> s >= 30000)                     // comparison
                  .map(s -> s + 5000)                          // MAP
                  .collect(Collectors.groupingBy(
                        s -> s >= 50000 ? "SENIOR" : "MID",
                        Collectors.summingInt(Integer::intValue) // REDUCE
                  ));

      System.out.println(result);
   }
}
```

---

4 filter() method call boolean + collect() (Map → Reduce → Store)

```
import java.util.*;
import java.util.stream.*;

public class FilterMethodMapReduceStore {

   static class User {
      String name;
      boolean active;

      User(String name, boolean active) {
         this.name = name;
         this.active = active;
      }

      boolean isActive() {
         return active;
      }
   }

   public static void main(String[] args) {
      List<User> users = List.of(
            new User("Anurag", true),
            new User("Amit", true),
            new User("Ravi", false)
      );

      Map<Character, Long> result =
            users.stream()
                  .filter(User::isActive)                    // method call
                  .map(u -> u.name)                          // MAP
                  .collect(Collectors.groupingBy(
                        name -> name.charAt(0),
                        Collectors.counting()                // REDUCE
                  ));

      System.out.println(result);
   }
}
```

---

5 distinct() equals/hashCode + collect() (Map → Reduce → Store)

```
import java.util.*;
import java.util.stream.*;

public class DistinctEqualsMapReduceStore {

   static class Person {
      String name;

      Person(String name) { this.name = name; }

      @Override
      public boolean equals(Object o) {
         return o instanceof Person p && name.equals(p.name);
      }

      @Override
      public int hashCode() {
         return Objects.hash(name);
      }
   }

   public static void main(String[] args) {
      List<Person> people = List.of(
            new Person("Anurag"),
            new Person("Anurag"),
            new Person("Ravi")
      );

      Map<Integer, Long> result =
            people.stream()
                  .distinct()
                  .map(p -> p.name)                    // MAP
                  .collect(Collectors.groupingBy(
                        String::length,
                        Collectors.counting()          // REDUCE
                  ));

      System.out.println(result);
   }
}
```

6 distinct() immutable objects + collect() (Map → Reduce → Store)

```
import java.util.*;
import java.util.stream.*;

public class DistinctImmutableMapReduceStore {
   public static void main(String[] args) {
      List<String> values = List.of("Java", "Java", "Go", "Python");

      Map<Character, Long> result =
            values.stream()
                  .distinct()
                  .map(String::toUpperCase)                    // MAP
                  .collect(Collectors.groupingBy(
                        s -> s.charAt(0),
                        Collectors.counting()                  // REDUCE
                  ));

      System.out.println(result);
   }
}
```

---

7 map() transform objects + collect() (Map → Reduce → Store)

```
import java.util.*;
import java.util.stream.*;

public class MapTransformMapReduceStore {
   public static void main(String[] args) {
      List<String> names = List.of("anurag", "amit", "ravi");

      Map<Integer, Long> result =
            names.stream()
                  .map(String::toUpperCase)                    // MAP
                  .collect(Collectors.groupingBy(
                        String::length,
                        Collectors.counting()                  // REDUCE
                  ));

      System.out.println(result);
   }
}
```

---

8 map() create new objects + collect() (Map → Reduce → Store)

```
import java.util.*;
import java.util.stream.*;

public class MapNewObjectMapReduceStore {

   static class UserDTO {
      String name;
      UserDTO(String name) { this.name = name; }
   }

   public static void main(String[] args) {
      List<String> names = List.of("Anurag", "Amit", "Ravi");

      Map<Character, Long> result =
            names.stream()
                  .map(UserDTO::new)                    // MAP
                  .collect(Collectors.groupingBy(
                        dto -> dto.name.charAt(0),
                        Collectors.counting()           // REDUCE
                  ));

      System.out.println(result);
   }
}
```

---

9 map() pure functions + collect() (Map → Reduce → Store)

```
import java.util.*;
import java.util.stream.*;

public class MapPureFunctionMapReduceStore {

   static int square(int n) {
      return n * n;
   }

   public static void main(String[] args) {
      List<Integer> numbers = List.of(2, 3, 4);

      Map<Boolean, Integer> result =
            numbers.stream()
                  .map(MapPureFunctionMapReduceStore::square)          // MAP
                  .collect(Collectors.groupingBy(
                        n -> n > 10,
                        Collectors.summingInt(Integer::intValue)       // REDUCE
                  ));

      System.out.println(result);
   }
}
```

---

10 flatMap() return Stream + collect() (Map → Reduce → Store)

```
import java.util.*;
import java.util.stream.*;

public class FlatMapStreamMapReduceStore {
   public static void main(String[] args) {
      List<List<Integer>> lists = List.of(
            List.of(1, 2),
            List.of(3, 4)
      );

      Map<Boolean, Integer> result =
            lists.stream()
                  .flatMap(List::stream)
                  .map(n -> n * 10)                          // MAP
                  .collect(Collectors.groupingBy(
                        n -> n > 20,
                        Collectors.summingInt(Integer::intValue)  // REDUCE
                  ));

      System.out.println(result);
   }
}
```

---

11 flatMap() split data + collect() (Map → Reduce → Store)

```
import java.util.*;
import java.util.stream.*;

public class FlatMapSplitMapReduceStore {
   public static void main(String[] args) {
      List<String> sentences = List.of("Java Stream", "Flat Map");

      Map<Integer, Long> result =
            sentences.stream()
                  .flatMap(s -> Arrays.stream(s.split(" ")))
                  .map(String::toLowerCase)                  // MAP
                  .collect(Collectors.groupingBy(
                        String::length,
                        Collectors.counting()                // REDUCE
                  ));
      System.out.println(result);
   }
}
```

---

12 flatMap() expand collections + collect() (Map → Reduce → Store)

```
import java.util.*;
import java.util.stream.*;

public class FlatMapExpandMapReduceStore {
   public static void main(String[] args) {
      Map<String, List<String>> data = Map.of(
            "A", List.of("x", "y"),
            "B", List.of("z")
      );

      Map<Integer, Long> result =
            data.values().stream()
                  .flatMap(Collection::stream)
                  .map(String::toUpperCase)                    // MAP
                  .collect(Collectors.groupingBy(
                        String::length,
                        Collectors.counting()                  // REDUCE
                  ));

      System.out.println(result);
   }
}
```

---

13 sorted() natural order + collect() (Map → Reduce → Store)

```
import java.util.*;
import java.util.stream.*;

public class SortedNaturalMapReduceStore {
   public static void main(String[] args) {
      List<Integer> numbers = List.of(5, 1, 3);

      Map<Boolean, Integer> result =
            numbers.stream()
                  .sorted()
                  .map(n -> n * 2)                             // MAP
                  .collect(Collectors.groupingBy(
                        n -> n > 5,
                        Collectors.summingInt(Integer::intValue)  // REDUCE
                  ));

      System.out.println(result);
   }
}
```

14 sorted() custom comparator + collect() (Map → Reduce → Store)

```
import java.util.*;
import java.util.stream.*;

public class SortedCustomMapReduceStore {
   public static void main(String[] args) {
      List<String> names = List.of("Anurag", "Ravi", "Amit");

      Map<Character, Long> result =
            names.stream()
                  .sorted(Comparator.reverseOrder())
                  .map(String::toUpperCase)                    // MAP
                  .collect(Collectors.groupingBy(
                        s -> s.charAt(0),
                        Collectors.counting()                  // REDUCE
                  ));

      System.out.println(result);
   }
}
```

15 sorted() multi-field + collect() (Map → Reduce → Store)

```
import java.util.*;
import java.util.stream.*;

public class SortedMultiFieldMapReduceStore {

   static class Employee {
      String name;
      int age;

      Employee(String name, int age) {
         this.name = name;
         this.age = age;
      }
   }

   public static void main(String[] args) {
      List<Employee> employees = List.of(
            new Employee("Anurag", 30),
            new Employee("Anurag", 25),
            new Employee("Ravi", 35)
      );

      Map<String, Integer> result =
            employees.stream()
                  .sorted(Comparator
                        .comparing((Employee e) -> e.name)
                        .thenComparingInt(e -> e.age))
                  .map(e -> e.age)                                  // MAP
                  .collect(Collectors.groupingBy(
                        age -> age >= 30 ? "SENIOR" : "JUNIOR",
                        Collectors.summingInt(Integer::intValue)  // REDUCE
                  ));

      System.out.println(result);
   }
}
```

16 peek() logging + collect() (Map → Reduce → Store)

```
import java.util.*;
import java.util.stream.*;

public class PeekLoggingMapReduceStore {
   public static void main(String[] args) {
      Map<Boolean, Integer> result =
            List.of(1, 2, 3).stream()
                  .peek(n -> System.out.println("Log: " + n))
                  .map(n -> n * 10)                                  // MAP
                  .collect(Collectors.groupingBy(
                        n -> n > 20,
                        Collectors.summingInt(Integer::intValue)  // REDUCE
                  ));

      System.out.println(result);
   }
}
```

---

17 peek() debugging + collect() (Map → Reduce → Store)

```
import java.util.*;
import java.util.stream.*;

public class PeekDebugMapReduceStore {
   public static void main(String[] args) {
      Map<Boolean, Long> result =
            List.of(10, 20, 30).stream()
                  .peek(n -> {
                     if (n == 20) System.out.println("Debug hit: " + n);
                  })
                  .map(n -> n + 1)                                   // MAP
                  .collect(Collectors.groupingBy(
                        n -> n > 20,
                        Collectors.counting()                        // REDUCE
                  ));

      System.out.println(result);
   }
}
```

---

18 peek() metrics + collect() (Map → Reduce → Store)

```
import java.util.*;
import java.util.concurrent.atomic.AtomicInteger;
import java.util.stream.*;

public class PeekMetricsMapReduceStore {
   public static void main(String[] args) {
      AtomicInteger counter = new AtomicInteger();

      Map<Boolean, Integer> result =
            List.of(5, 10, 15).stream()
                  .peek(n -> counter.incrementAndGet())
                  .map(n -> n * 2)                                    // MAP
                  .collect(Collectors.groupingBy(
                        n -> n > 20,
                        Collectors.summingInt(Integer::intValue) // REDUCE
                  ));
      System.out.println("Processed: " + counter.get());
      System.out.println(result);
   }
}
```